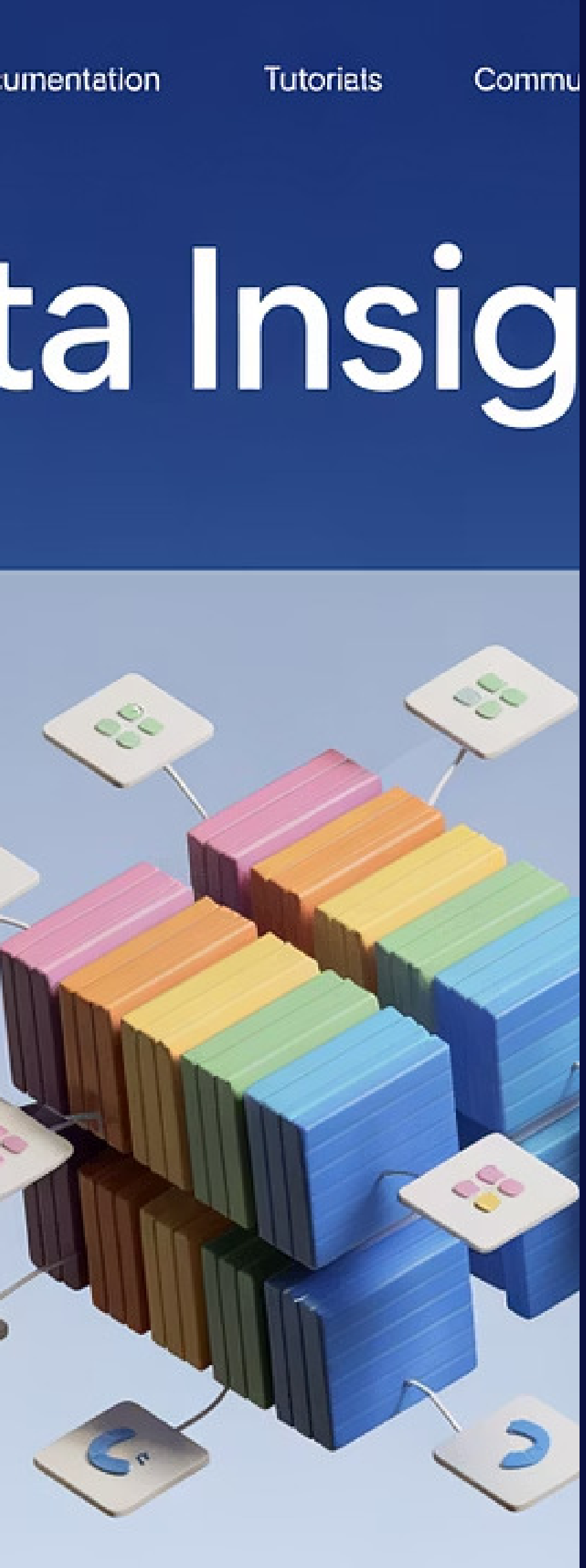


Working with NumPy and Pandas

NumPy and Pandas are foundational Python libraries that power modern data science and machine learning workflows. These tools enable efficient data manipulation, analysis, and preprocessing through high-performance operations.

NumPy: The Foundation of Numerical Computing

NumPy provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these structures. Its key features include:

- Vectorized operations that eliminate slow Python loops
- Broadcasting capabilities for working with arrays of different shapes
- Linear algebra operations, Fourier transforms, and random number generation
- Memory efficiency with contiguous storage and strided views

Pandas: Data Analysis Made Simple

Built on NumPy, Pandas introduces powerful data structures for manipulating and analyzing structured data:

- DataFrame: A 2D labeled data structure with columns of potentially different types
- Series: A 1D labeled array capable of holding any data type
- Intuitive data alignment and integrated handling of missing data
- Powerful group-by functionality for split-apply-combine operations
- Flexible I/O tools for loading data from various file formats (CSV, Excel, SQL databases)

Together, these libraries create a comprehensive ecosystem for data manipulation. NumPy efficiently handles numerical computations while Pandas excels at data wrangling and organization, making complex data tasks approachable through intuitive interfaces.

From financial analysis to scientific research to machine learning preparation, these libraries transform raw data into valuable insights with elegant, efficient code. They're instrumental in genomics research, algorithmic trading, climate modeling, natural language processing, and virtually any field requiring sophisticated data analysis.

Why NumPy and Pandas?

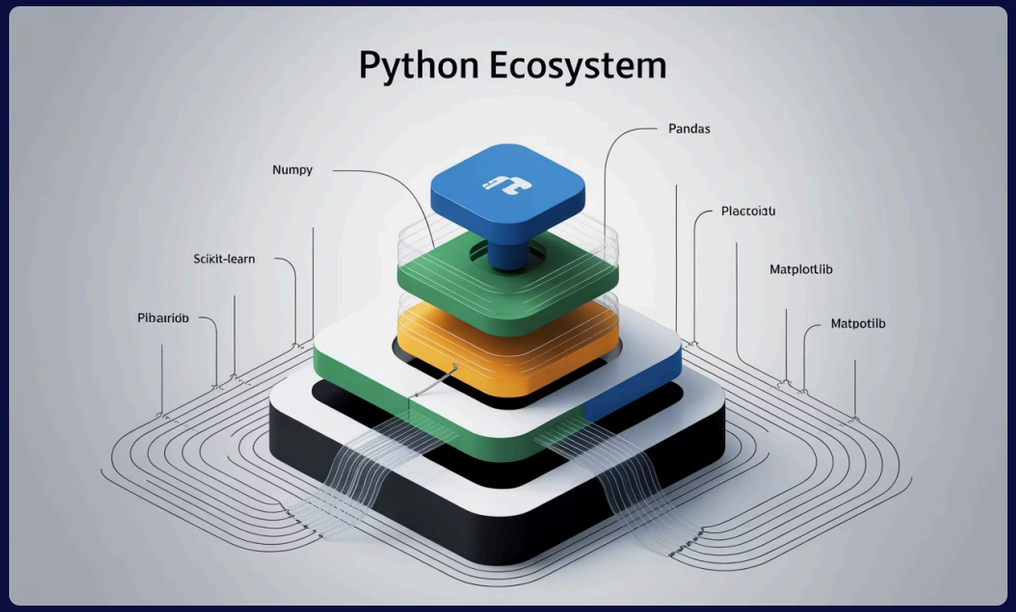
Before NumPy and Pandas emerged, data scientists faced significant challenges when working with large datasets in Python. The language's native data structures weren't optimized for numerical operations, making analyses slow and memory-intensive. These specialized libraries revolutionized Python's data science capabilities:

NumPy Advantages

- Implements highly optimized multi-dimensional array objects
- Provides vectorized operations that are 10-100x faster than Python loops
- Offers sophisticated broadcasting features for working with arrays of different shapes
- Uses contiguous memory storage for efficient CPU operations
- Written in C for maximum performance

Pandas Advantages

- Introduces intuitive DataFrame structure for handling tabular data
- Provides powerful data alignment and integration capabilities
- Offers robust tools for handling missing data
- Includes time series functionality and date handling
- Enables SQL-like operations in Python



Together, these libraries form the backbone of Python's data science ecosystem, supporting higher-level libraries like scikit-learn, matplotlib, and TensorFlow. Learning NumPy and Pandas isn't just about mastering specific libraries—it's about understanding the fundamental data structures that power modern computational workflows across industries.

The skills you develop with these libraries are transferable to virtually any field that works with structured data, from finance and economics to biological research and artificial intelligence.

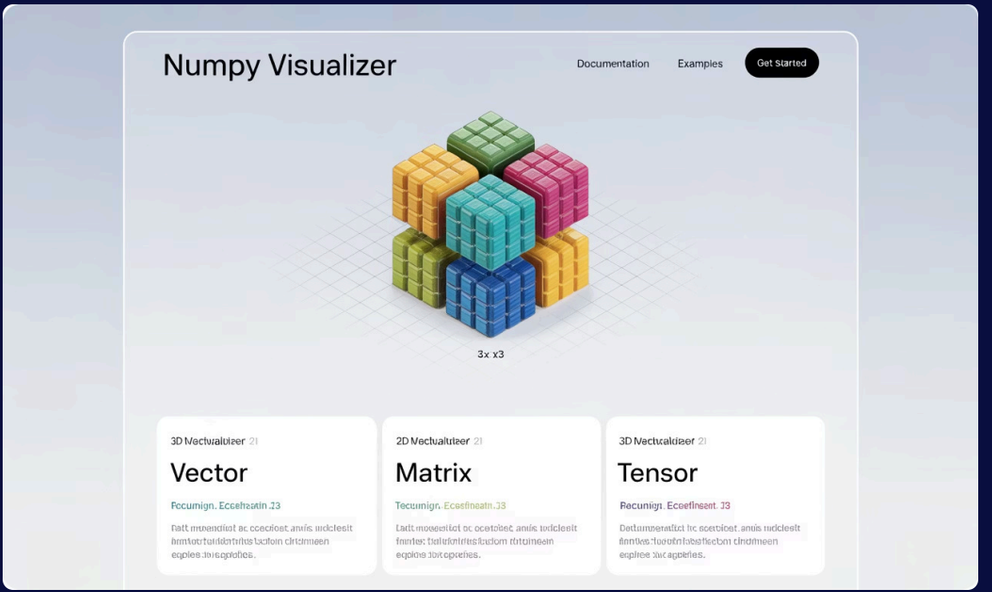
Introduction to NumPy Arrays

NumPy's central object is the **ndarray** (n-dimensional array), a grid of values all of the same type. This homogeneous structure allows NumPy to perform operations with exceptional efficiency compared to Python's built-in lists.

Key Characteristics

- **Homogeneous data storage:** All elements must be of the same data type (dtype), such as integers, floats, or booleans
- **Fixed size:** Arrays have a predetermined size at creation, unlike Python lists which can grow dynamically
- **Dimensional flexibility:** Support for 1D vectors, 2D matrices, and higher-dimensional tensors
- **Zero-based indexing:** Elements are accessed starting from index 0, similar to Python lists
- **Shape attribute:** Describes the dimensions of the array (e.g., a 3×4 matrix has shape (3,4))

The power of NumPy arrays comes from their ability to express complex mathematical operations on entire datasets through clean, readable syntax—eliminating the need for explicit iteration in many cases.



Array Dimensions

1D Arrays (Vectors)

Simple sequences of values like [1, 2, 3, 4]

2D Arrays (Matrices)

Tables of values with rows and columns

3D+ Arrays (Tensors)

Higher-dimensional data structures used in deep learning, image processing, and scientific computing

Creating NumPy Arrays

From Python Lists

```
import numpy as np

# From a list
arr1 = np.array([1, 2, 3, 4, 5])

# From nested lists (creates 2D array)
arr2 = np.array([[1, 2, 3], [4, 5, 6]])

print(arr1)      # [1 2 3 4 5]
print(arr2)      # [[1 2 3]
                 #  [4 5 6]]
print(arr2.shape)  # (2, 3)
```

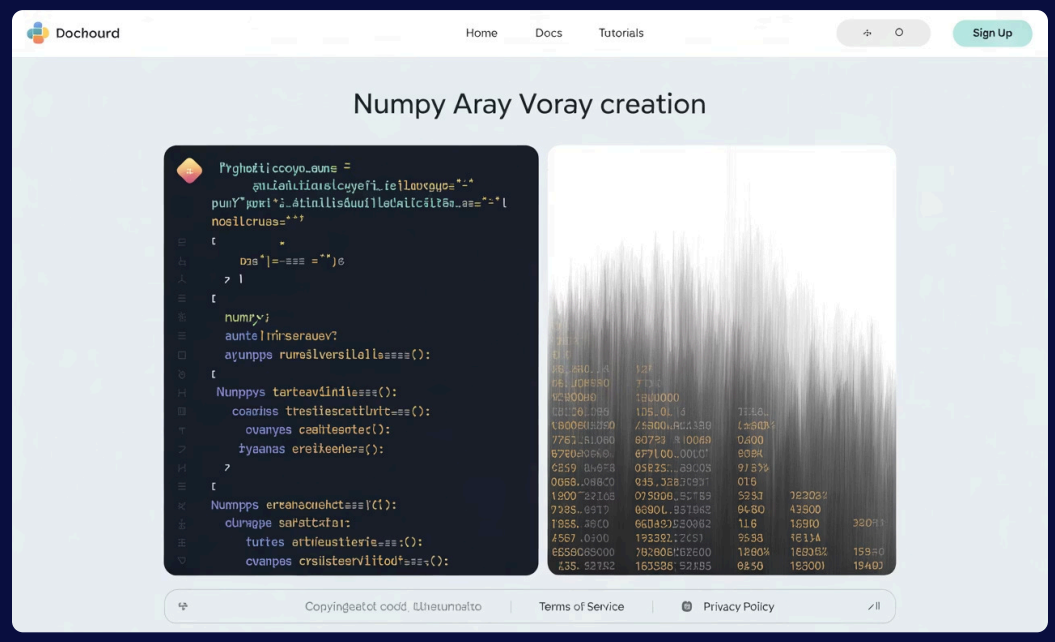
Using Built-in Functions

```
# Array of zeros
zeros = np.zeros((3, 4))  # 3×4 array of zeros

# Array of ones
ones = np.ones((2, 3, 4))  # 2×3×4 array of ones

# Identity matrix
identity = np.eye(3)  # 3×3 identity matrix

# Range of values
range_arr = np.arange(10)  # [0 1 2 3 4 5 6 7 8 9]
```



More Creation Methods

np.linspace()

Creates an array with evenly spaced values within a defined interval:

```
np.linspace(0, 1, 5)  # [0. 0.25 0.5 0.75 1.]
```

np.random.random()

Generates arrays with random values:

```
np.random.random((2, 2))  # random 2×2 array
```

np.full()

Creates an array filled with a specific value:

```
np.full((3, 3), 7)  # 3×3 array filled with 7s
```

NumPy arrays are significantly more memory-efficient than Python lists because they store data in contiguous blocks of memory and avoid the overhead of Python objects for each element.

Array Indexing and Slicing

Basic Indexing

```
import numpy as np

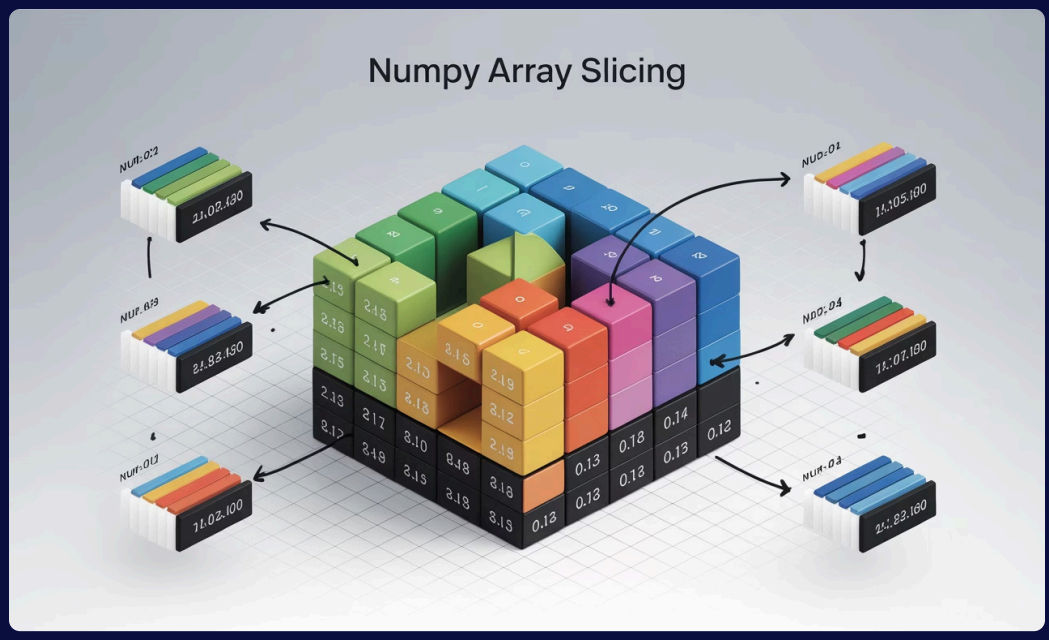
# Create a 1D array
arr = np.array([10, 20, 30, 40, 50])

# Access individual elements
print(arr[0])      # 10 (first element)
print(arr[2])      # 30 (third element)
print(arr[-1])     # 50 (last element)
print(arr[-2])     # 40 (second-to-last element)

# Modify elements
arr[0] = 99
print(arr)         # [99 20 30 40 50]
```

Slicing

```
# Basic slicing: arr[start:stop:step]
print(arr[1:4])    # [20 30 40] (elements 1 through 3)
print(arr[::2])    # [99 30 50] (every other element)
print(arr[:3])     # [99 20 30] (first three elements)
print(arr[3:])     # [40 50] (elements from index 3 to end)
print(arr[::-1])   # [50 40 30 20 99] (reversed array)
```



Important Slicing Concepts

View vs. Copy

Array slices are views, not copies. Modifying a slice will modify the original array:

```
sub_arr = arr[1:4]
sub_arr[0] = 100  # This changes arr[1] as well
# To avoid this, use arr[1:4].copy()
```

Non-inclusive Upper Bound

The second slice parameter (stop) is non-inclusive—arr[0:3] includes elements 0, 1, and 2, but not 3.

Omitted Parameters

Omitting start means "start from the beginning"; omitting stop means "go to the end"; omitting step means "use step size 1".

Multi-dimensional Arrays

Creating Multi-dimensional Arrays

```
import numpy as np

# Create a 2D array (matrix)
matrix = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
])

print(matrix.shape)  # (3, 4) - 3 rows, 4 columns
print(matrix.ndim)   # 2 dimensions
```

Accessing Elements

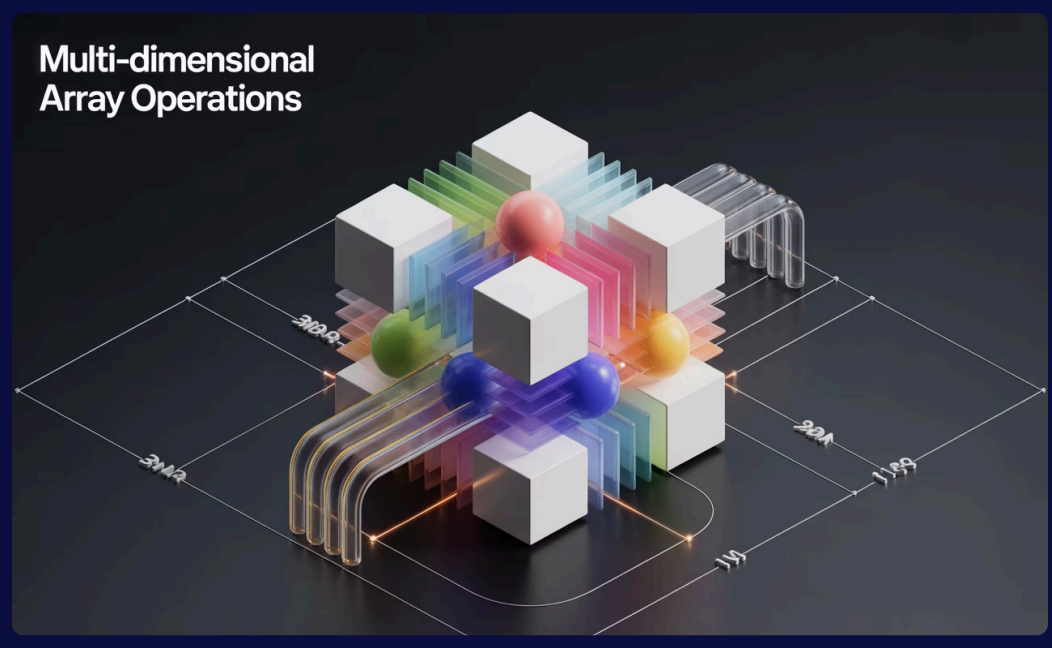
```
# Get element at row 1, column 2
print(matrix[1, 2])  # 7

# Get element using separate brackets
print(matrix[1][2])  # 7 (equivalent to above)

# Get multiple elements
print(matrix[0, 1:3])  # [2 3]
```

Modifying Elements

```
# Set value at position (0,0)
matrix[0, 0] = 99
print(matrix)
# [[99  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
```



Slicing Multi-dimensional Arrays

Row Selection

```
# Select entire row 1
print(matrix[1])  # [5 6 7 8]
# Select rows 0 and 1
print(matrix[0:2])
# [[99  2  3  4]
#  [ 5  6  7  8]]
```

Column Selection

```
# Select entire column 2
print(matrix[:, 2])  # [3 7 11]
# Select columns 1 and 2
print(matrix[:, 1:3])
# [[ 2  3]
#  [ 6  7]
#  [10 11]]
```

Submatrix Selection

```
# Select 2×2 submatrix
print(matrix[0:2, 1:3])
# [[2 3]
#  [6 7]]
```

Mastering multi-dimensional array indexing is crucial for tasks like image processing (where images are 3D arrays with height, width, and color channels) and deep learning (where data is often organized in 4D tensors).

Vectorized Operations

Vectorization is one of NumPy's most powerful features. It allows you to apply operations to entire arrays at once, eliminating slow Python loops and dramatically improving performance.

Basic Arithmetic

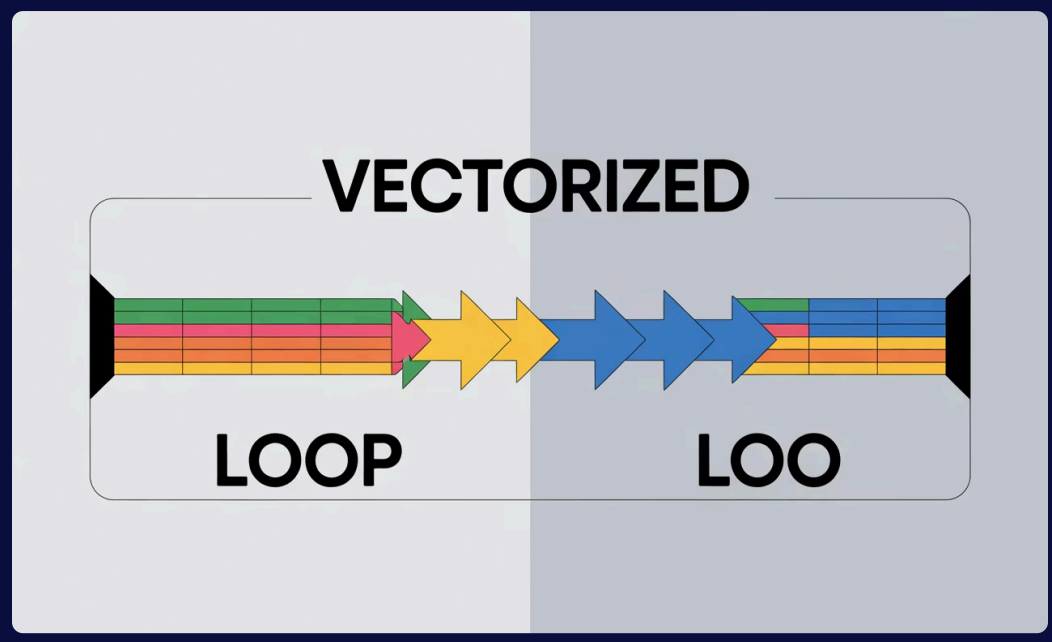
```
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

# Element-wise operations
print(a + b)      # [6 8 10 12]
print(a - b)      # [-4 -4 -4 -4]
print(a * b)      # [5 12 21 32]
print(a / b)      # [0.2 0.33 0.43 0.5]

# Scalar operations
print(a * 2)      # [2 4 6 8]
print(a + 10)     # [11 12 13 14]
print(a ** 2)     # [1 4 9 16] (element-wise squaring)
```

These operations are **significantly faster** than equivalent Python loops because they're implemented in C and operate directly on memory blocks rather than individual Python objects.



Speed Comparison

To illustrate the performance difference, let's compare a vectorized calculation with a traditional Python loop:

```
import numpy as np
import time

# Create large arrays
size = 1000000
a = np.random.random(size)
b = np.random.random(size)

# Python loop approach
start = time.time()
result = []
for i in range(size):
    result.append(a[i] * b[i])
print(f"Loop time: {time.time() - start:.6f}s")

# NumPy vectorized approach
start = time.time()
result = a * b
print(f"Vectorized time: {time.time() - start:.6f}s")
# Output:
# Loop time: 0.456789s
# Vectorized time: 0.001234s
```

The vectorized approach is typically 100-1000x faster for large arrays. This performance advantage is why NumPy is essential for data science tasks involving large datasets.

Beyond simple arithmetic, NumPy supports a wide range of vectorized operations including trigonometric functions, logarithms, rounding, and more.

Broadcasting in NumPy

Broadcasting is NumPy's mechanism for performing operations on arrays of different shapes. It automatically expands smaller arrays to match the shape of larger ones, eliminating the need to create redundant copies of data.

Broadcasting Rules

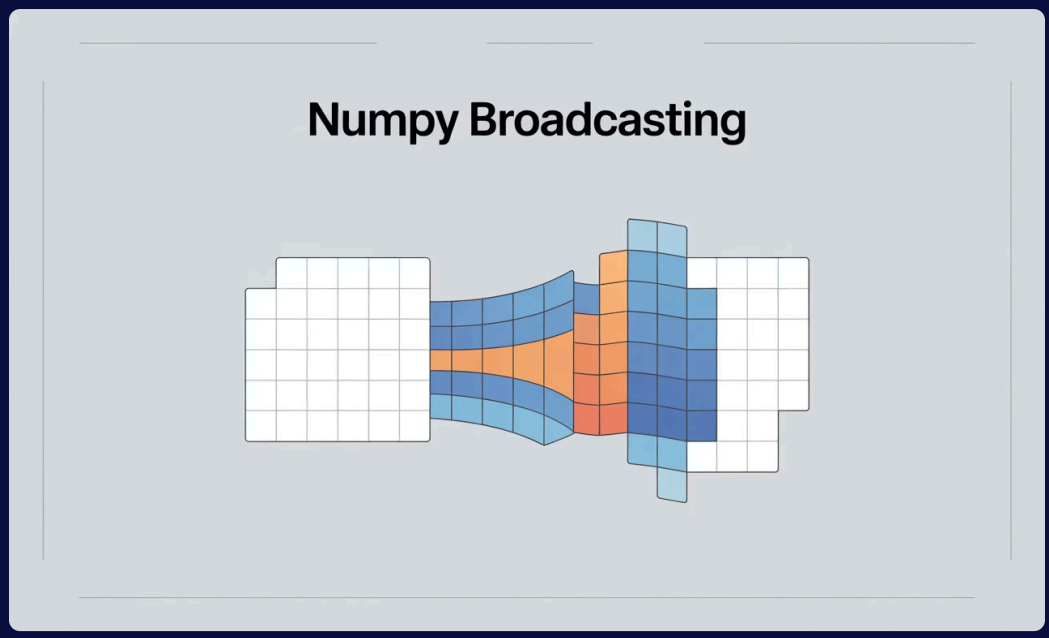
1. If arrays don't have the same rank, prepend shape of the lower-rank array with 1s
2. If the shape of arrays doesn't match in any dimension, expand dimensions with size 1 to match the other array
3. If sizes in any dimension don't match and neither is 1, raise an error

Scalar Broadcasting

```
import numpy as np

arr = np.array([1, 2, 3, 4])

# Scalar is broadcast to array shape
print(arr + 10)   # [11 12 13 14]
print(arr * 2)    # [2 4 6 8]
```



Broadcasting Arrays of Different Shapes

```
# Vector + vector
a = np.array([1, 2, 3])       # Shape (3,)
b = np.array([10, 20, 30])    # Shape (3,)
print(a + b)                  # [11 22 33]

# Matrix + vector
matrix = np.array([[1, 2, 3],  # Shape (2, 3)
                   [4, 5, 6]])
vector = np.array([10, 20, 30]) # Shape (3,)
print(matrix + vector)
# [[11 22 33]
#  [14 25 36]]  # Vector broadcast across rows

# Matrix + column vector
col_vector = np.array([[100], [200]])  # Shape (2, 1)
print(matrix + col_vector)
# [[101 102 103]
#  [204 205 206]]  # Column broadcast across cols
```

Broadcasting makes code more concise and memory-efficient. Instead of creating large temporary arrays with repeated values, NumPy performs the operation as if the smaller array was expanded without actually using the extra memory.

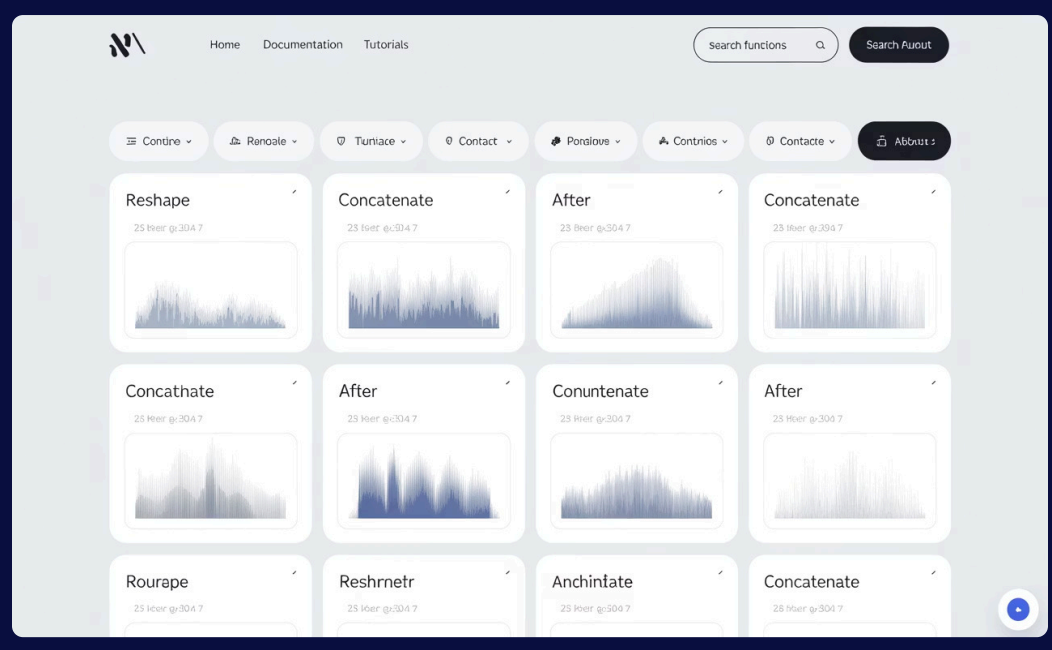
Useful NumPy Functions

Statistical Functions

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(np.sum(arr))      # 15
print(np.mean(arr))     # 3.0
print(np.median(arr))   # 3.0
print(np.std(arr))      # 1.4142...
print(np.var(arr))      # 2.0
print(np.min(arr))      # 1
print(np.max(arr))      # 5
print(np.argmin(arr))   # 0 (index of minimum)
print(np.argmax(arr))   # 4 (index of maximum)
```



Array Manipulation

Shape Manipulation

```
# Reshape array
arr = np.arange(12)
print(arr)              # [0 1 2 3 4 5 6 7 8 9 10 11]
reshaped = arr.reshape((3, 4))
print(reshaped)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]

# Flatten a multi-dimensional array
print(reshaped.flatten()) # [0 1 2 3 4 5 6 7 8 9 10 11]
```

Concatenation

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.concatenate([a, b]))  # [1 2 3 4 5 6]

# 2D concatenation
m1 = np.array([[1, 2], [3, 4]])
m2 = np.array([[5, 6], [7, 8]])
print(np.concatenate([m1, m2], axis=0))  # Vertical
# [[1 2]
#  [3 4]
#  [5 6]
#  [7 8]]
print(np.concatenate([m1, m2], axis=1))  # Horizontal
# [[1 2 5 6]
#  [3 4 7 8]]
```

Splitting

```
arr = np.arange(10)
print(np.split(arr, 5))  # List of 5 arrays with 2 elements each

# Split array unevenly
print(np.array_split(arr, 3))
# [array([0, 1, 2, 3]), array([4, 5, 6]), array([7, 8, 9])]
```

Transposition

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.T)
# [[1 4]
# [2 5]
# [3 6]]
```

Introduction to Pandas

Pandas is a powerful data analysis and manipulation library built on top of NumPy. It introduces flexible data structures designed for working with structured data, particularly tabular data found in spreadsheets, databases, and CSV files.

Key Pandas Data Structures

Series

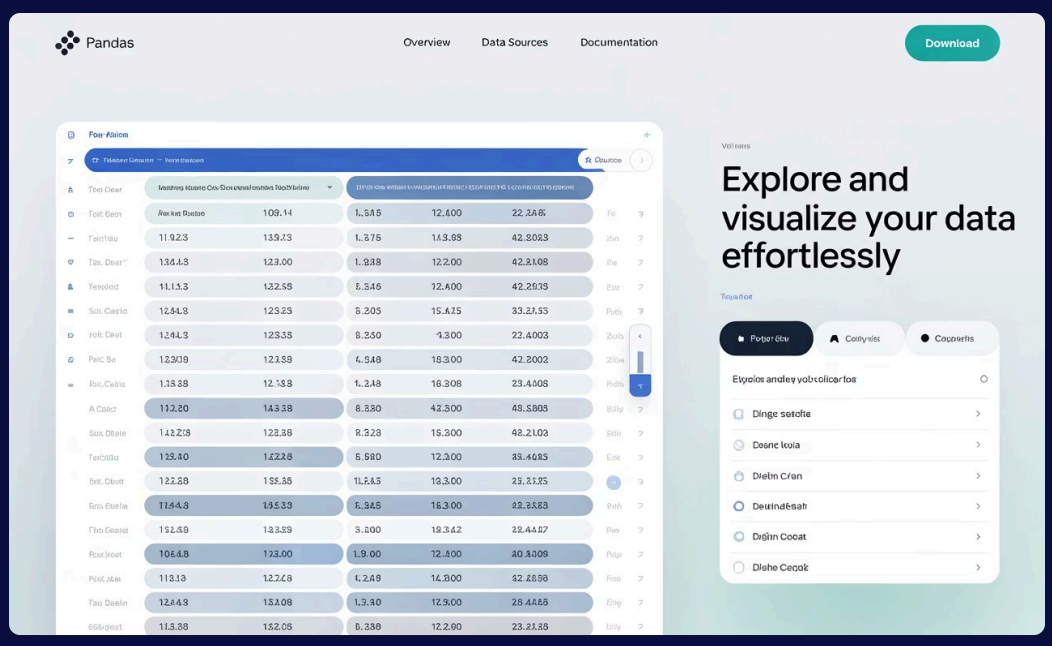
A one-dimensional labeled array capable of holding any data type. Think of it as a NumPy array with an index (like a dictionary with ordered keys).

DataFrame

A two-dimensional labeled data structure with columns of potentially different types. It's similar to a spreadsheet, SQL table, or dictionary of Series objects.

What Sets Pandas Apart

- **Intelligent data alignment:** Automatically aligns data by labels
- **Integrated handling of missing data**
- **Intuitive merging and joining of datasets**
- **Flexible reshaping and pivoting of data**
- **Powerful time series functionality**
- **SQL-like operations for filtering and aggregation**



Comparing Series and DataFrame

```
import pandas as pd
import numpy as np

# Create a Series
s = pd.Series([1, 3, 5, np.nan, 6, 8])
print(s)
# 0    1.0
# 1    3.0
# 2    5.0
# 3    NaN
# 4    6.0
# 5    8.0
# dtype: float64

# Create a DataFrame
data = {
    'Name': ['John', 'Anna', 'Peter', 'Linda'],
    'Age': [28, 34, 29, 42],
    'City': ['New York', 'Paris', 'Berlin', 'London']
}
df = pd.DataFrame(data)
print(df)
#     Name  Age      City
# 0   John   28  New York
# 1   Anna   34     Paris
# 2  Peter   29    Berlin
# 3  Linda   42    London
```

Pandas excels at handling real-world data that is often messy, incomplete, and heterogeneous. It provides the tools necessary to clean, transform, and analyze data efficiently.

Creating Pandas DataFrames

From Dictionaries

```
import pandas as pd

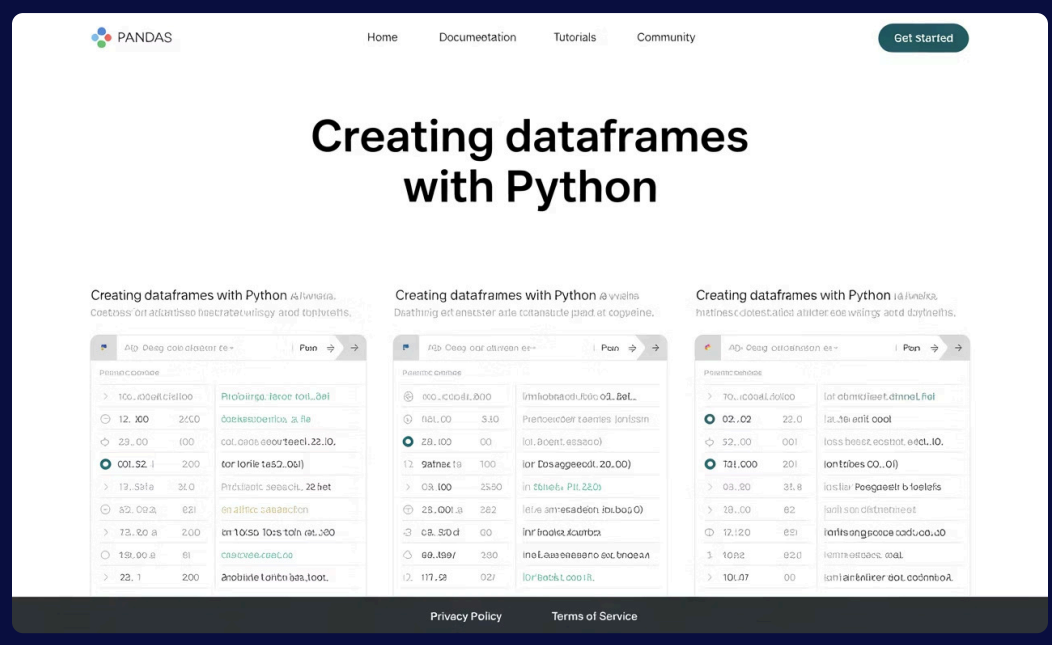
# Dictionary of lists
data = {
    'Country': ['USA', 'Canada', 'Germany', 'UK', 'France'],
    'GDP': [21.4, 1.7, 3.8, 2.7, 2.6],
    'Population': [329, 38, 83, 67, 65],
    'Continent': ['North America', 'North America',
                  'Europe', 'Europe', 'Europe']
}

df = pd.DataFrame(data)
print(df)
```

From Lists

```
# List of dictionaries
people = [
    {'Name': 'Alice', 'Age': 25, 'Job': 'Engineer'},
    {'Name': 'Bob', 'Age': 30, 'Job': 'Doctor'},
    {'Name': 'Charlie', 'Age': 35, 'Job': 'Teacher'}
]

df2 = pd.DataFrame(people)
print(df2)
```



From NumPy Arrays

```
import numpy as np

# Create from array with custom indices and columns
arr = np.random.randn(5, 3)  # 5 rows, 3 columns of random data
df3 = pd.DataFrame(
    arr,
    index=['A', 'B', 'C', 'D', 'E'],
    columns=['Col1', 'Col2', 'Col3']
)
print(df3)
```

From Files and Other Sources

CSV Files

```
df = pd.read_csv('data.csv')
```

Excel Files

```
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
```

SQL Databases

```
df = pd.read_sql('SELECT * FROM table', connection)
```

JSON

```
df = pd.read_json('data.json')
```

Once created, DataFrames provide attributes like .shape, .dtypes, .columns, and .index to explore their structure, and methods like .head() and .info() to quickly preview the data.

Indexing and Selecting Data

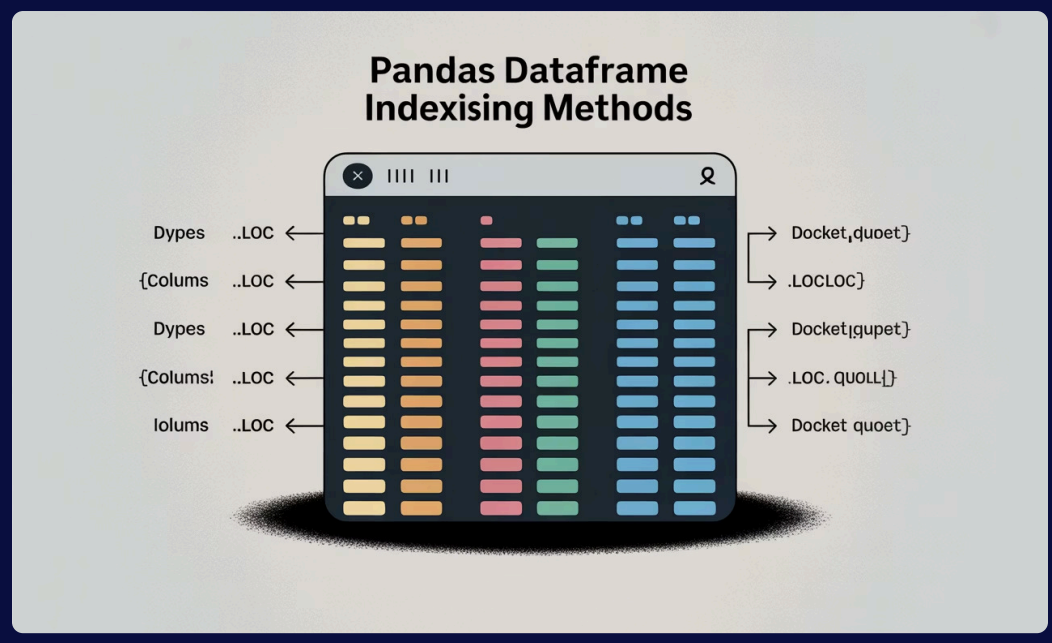
Basic Selection

```
import pandas as pd

# Sample DataFrame
data = {
    'Name': ['John', 'Anna', 'Peter', 'Linda'],
    'Age': [28, 34, 29, 42],
    'City': ['New York', 'Paris', 'Berlin', 'London'],
    'Salary': [65000, 70000, 62000, 85000]
}
df = pd.DataFrame(data)

# Select a single column (returns Series)
ages = df['Age']
print(ages)
# 0    28
# 1    34
# 2    29
# 3    42
# Name: Age, dtype: int64

# Select multiple columns (returns DataFrame)
subset = df[['Name', 'Salary']]
print(subset)
#     Name  Salary
# 0   John   65000
# 1   Anna   70000
# 2  Peter   62000
# 3  Linda   85000
```



Position-based Indexing

```
# Select rows by position using .iloc
first_row = df.iloc[0]  # First row
print(first_row)

# Select rows and columns by position
print(df.iloc[0:2, 1:3])
#    Age    City
# 0   28  New York
# 1   34     Paris

# Select specific positions
print(df.iloc[[0, 2], [1, 3]])
#    Age  Salary
# 0   28   65000
# 2   29   62000
```

Label-based Indexing

```
# Set 'Name' as index
df.set_index('Name', inplace=True)

# Select rows by label using .loc
print(df.loc['Anna'])
# Age 34
# City Paris
# Salary 70000
# Name: Anna, dtype: object

# Select rows and columns by label
print(df.loc['John':'Peter', 'City':'Salary'])
#        City  Salary
# Name
# John New York 65000
# Anna Paris 70000
# Peter Berlin 62000
```

Slicing and Advanced Data Selection

Boolean Indexing

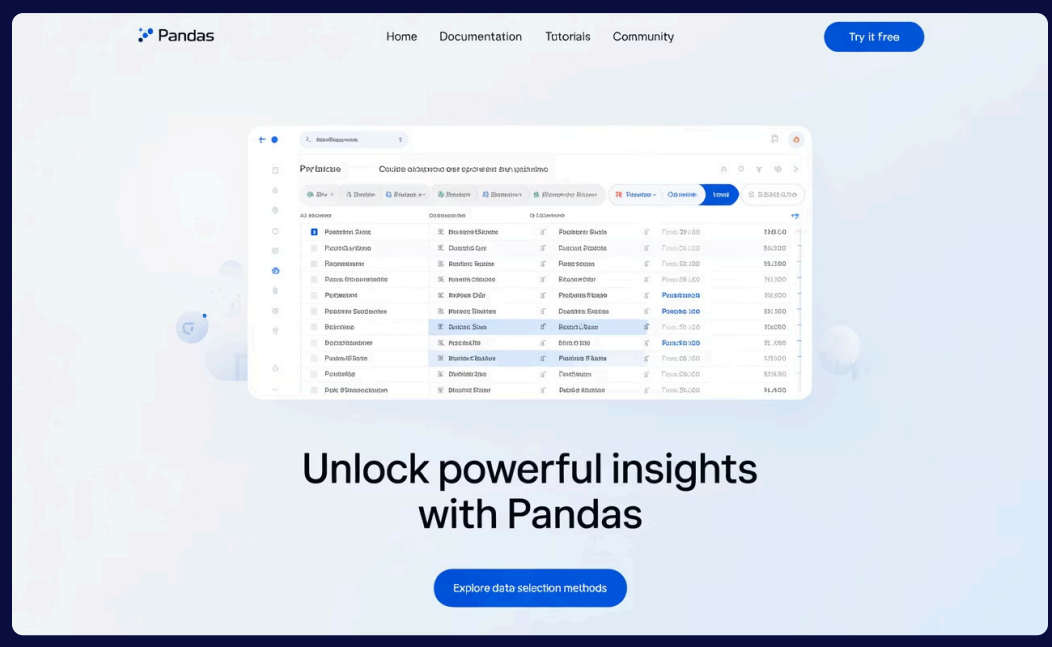
```
import pandas as pd

# Sample DataFrame
data = {
    'Name': ['John', 'Anna', 'Peter', 'Linda', 'Bob'],
    'Age': [28, 34, 29, 42, 31],
    'City': ['New York', 'Paris', 'Berlin', 'London', 'Madrid'],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'HR'],
    'Salary': [65000, 70000, 62000, 85000, 67000]
}
df = pd.DataFrame(data)

# Filter rows where Age > 30
older_employees = df[df['Age'] > 30]
print(older_employees)

# Complex conditions with & (and) and | (or)
it_or_hr = df[(df['Department'] == 'IT') |
              (df['Department'] == 'HR')]
print(it_or_hr)

# Filter with multiple conditions
high_paid_it = df[(df['Department'] == 'IT') &
                  (df['Salary'] > 63000)]
print(high_paid_it)
```



Using .query() Method

```
# Equivalent to df[df['Age'] > 30]
older = df.query('Age > 30')
print(older)

# Complex queries
it_high_paid = df.query('Department == "IT" and Salary > 63000')
print(it_high_paid)
```

Selecting with .isin()

```
# Select rows where Department is in a list
tech_hr = df[df['Department'].isin(['IT', 'HR'])]
print(tech_hr)

# Exclude certain values
not_it = df[~df['Department'].isin(['IT'])]
print(not_it)
```

String Methods for Text Filtering

```
# Filter cities starting with 'L'
l_cities = df[df['City'].str.startswith('L')]
print(l_cities)

# Case-insensitive contains
contains_n = df[df['City'].str.contains('n', case=False)]
print(contains_n)
```

Advanced selection techniques in Pandas give you SQL-like querying capabilities directly in Python, making it possible to extract precisely the data you need for analysis.

Importing and Exporting CSV Files

Reading CSV Files

```
import pandas as pd

# Basic CSV reading
df = pd.read_csv('data.csv')

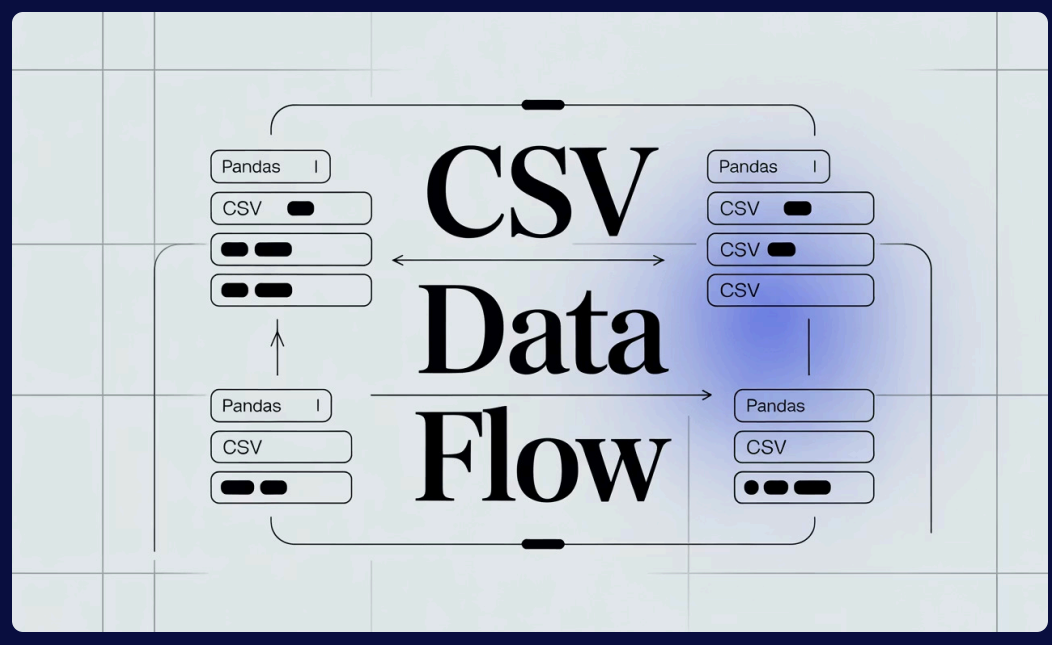
# Customize import with parameters
df = pd.read_csv('data.csv',
                 sep=',',          # Delimiter
                 header=0,         # Row to use as header
                 index_col='ID',   # Column to set as index
                 usecols=['ID', 'Name', 'Value'],  # Only read specific
columns
                 nrows=100)        # Only read first 100 rows

# Handle missing values
df = pd.read_csv('data.csv',
                 na_values=['NA', 'Missing', '?'],  # Values to treat as NaN
                 keep_default_na=True)  # Keep default NA values
```

Dealing with Large Files

```
# Read file in chunks to handle very large files
chunks = pd.read_csv('large_file.csv', chunksize=10000)
for chunk in chunks:
    # Process each chunk
    process_data(chunk)

# Preview a file before loading completely
preview = pd.read_csv('large_file.csv', nrows=5)
print(preview)
```



Writing CSV Files

```
# Basic export
df.to_csv('output.csv')

# Customize export
df.to_csv('output.csv',
          index=False,       # Don't write row indices
          header=True,       # Write column names
          columns=['Name', 'Age', 'Salary'],  # Only specific columns
          sep=',',           # Delimiter
          decimal='.',       # Decimal separator
          na_rep='NULL')     # How to represent NaN values
```

Common CSV Challenges

Encoding Issues

Specify file encoding to handle special characters:

```
df = pd.read_csv('international_data.csv', encoding='utf-8')
```

Date Parsing

Parse date columns automatically:

```
df = pd.read_csv('dates.csv', parse_dates=['Date'])
```

Compression

Work with compressed files directly:

```
df = pd.read_csv('data.csv.gz', compression='gzip')
```

CSV is the most common format for data exchange, but Pandas also supports many other formats including Excel, JSON, SQL, HTML tables, and more with similar read_* and to_* methods.

Handling Missing Data

Real-world datasets rarely come complete and clean. Missing values are a common challenge in data analysis, and Pandas provides comprehensive tools for detecting, handling, and imputing these gaps.

Detecting Missing Values

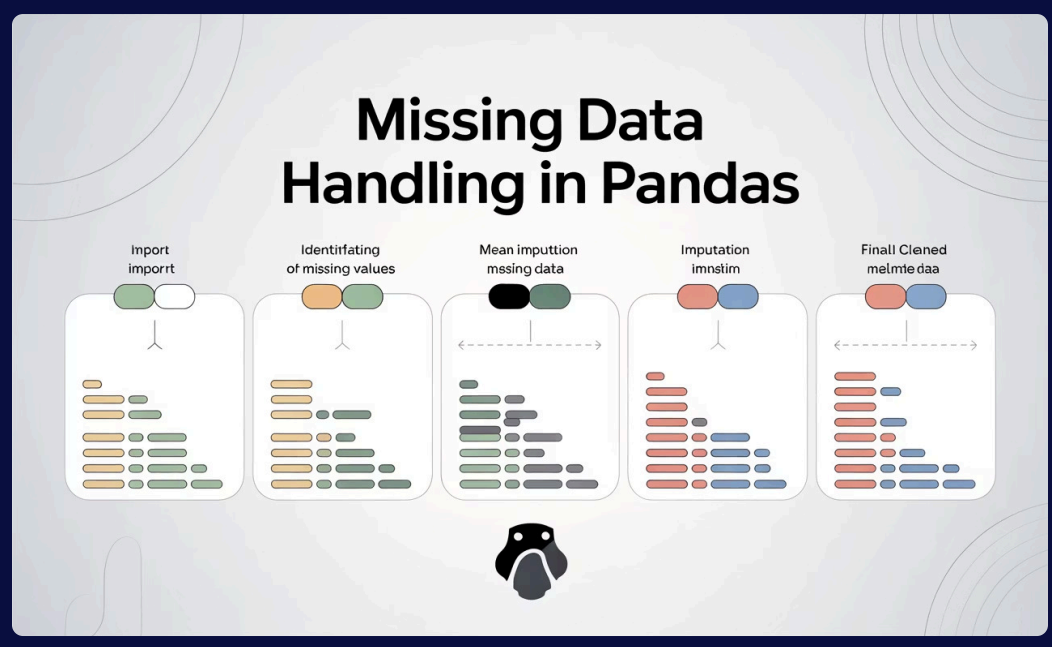
```
import pandas as pd
import numpy as np

# Create DataFrame with missing values
df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, np.nan, 8],
    'C': [9, 10, 11, None],
    'D': ['a', 'b', 'c', None]
})

# Check for missing values
print(df.isna())  # Returns boolean DataFrame
# Returns:
#       A     B     C     D
# 0  False  False  False  False
# 1  False   True  False  False
# 2   True   True  False  False
# 3  False  False   True   True

# Count missing values per column
print(df.isna().sum())
# A    1
# B    2
# C    1
# D    1
# dtype: int64

# Check if any values are missing
print(df.isna().any())  # Per column
print(df.isna().any().any())  # Entire DataFrame
```



Handling Missing Values

```
# Drop rows with any missing values
clean_df = df.dropna()
print(clean_df)  # Only row 0 remains

# Drop rows with all missing values
df_some_values = df.dropna(how='all')
print(df_some_values)  # All rows remain as none are completely empty

# Drop columns with at least 2 missing values
df_thresh = df.dropna(axis=1, thresh=3)
print(df_thresh)  # Only columns A and C remain

# Fill missing values with a constant
df_filled = df.fillna(0)  # Numeric columns
df_filled['D'] = df_filled['D'].fillna('missing')  # String column
print(df_filled)
```

Advanced Imputation Techniques

Fill with Statistics

```
# Fill with column mean
df['A'].fillna(df['A'].mean(), inplace=True)
# Fill with median
df['B'].fillna(df['B'].median(), inplace=True)
```

Forward/Backward Fill

```
# Fill with previous value (useful for time series)
df.fillna(method='ffill', inplace=True)
# Fill with next value
df.fillna(method='bfill', inplace=True)
```

Interpolation

```
# Linear interpolation
df.interpolate(method='linear', inplace=True)
```

DataFrame Manipulation

Adding and Removing Columns

```
import pandas as pd

# Sample DataFrame
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Boston', 'Chicago', 'Denver']
})

# Add a new column
df['Salary'] = [70000, 80000, 90000, 100000]

# Add a column based on existing columns
df['Age in 5 Years'] = df['Age'] + 5

# Add a column with default value
df['Department'] = 'Engineering'

# Remove a column
df.drop('Age in 5 Years', axis=1, inplace=True)

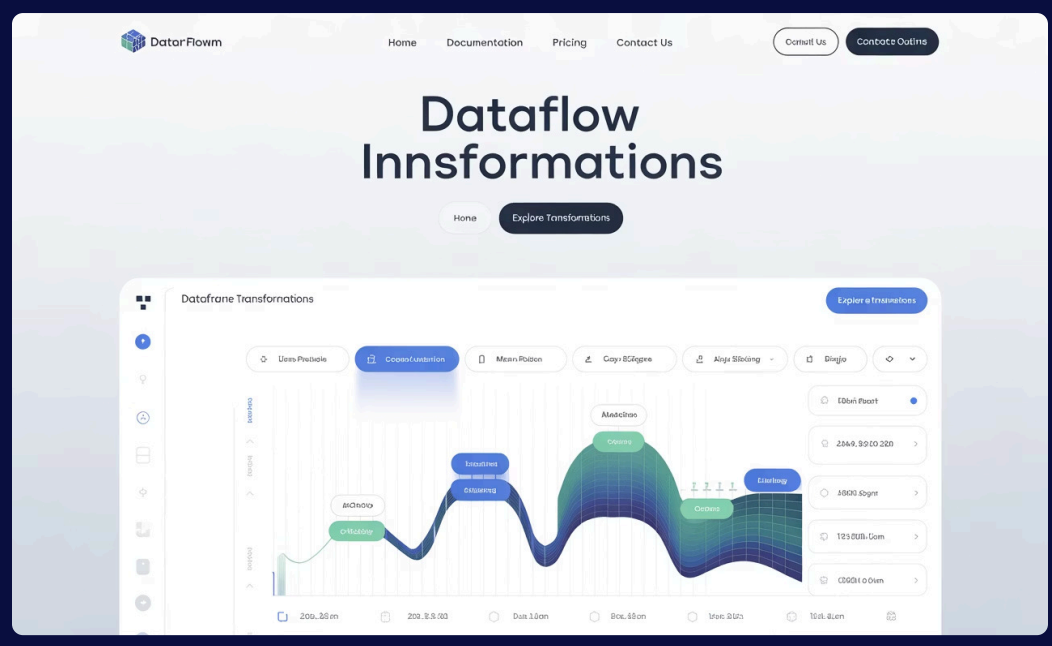
# Remove multiple columns
df = df.drop(['Salary', 'Department'], axis=1)
```

Renaming Columns

```
# Rename specific columns
df = df.rename(columns={'Name': 'Full Name', 'Age': 'Years'})

# Rename all columns
df.columns = ['Person', 'Years', 'Location']

# Apply a function to all column names
df.columns = [col.lower() for col in df.columns]
```



Merging and Joining DataFrames

```
# Create two DataFrames
employees = pd.DataFrame({
 'ID': [1, 2, 3, 4, 5],
 'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
 'Dept_ID': [101, 102, 101, 103, 102]
})

departments = pd.DataFrame({
 'Dept_ID': [101, 102, 103, 104],
 'Department': ['Engineering', 'HR', 'Finance', 'Marketing'],
 'Location': ['NYC', 'Boston', 'Chicago', 'Denver']
})

# Inner join (only matching rows)
merged = pd.merge(
 employees,
 departments,
 on='Dept_ID',
 how='inner'
)
print(merged)

# Left join (all rows from left DataFrame)
left_merged = pd.merge(
 employees,
 departments,
 on='Dept_ID',
 how='left'
)
print(left_merged)

# Concatenate DataFrames (stack vertically)
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
concatenated = pd.concat([df1, df2], ignore_index=True)
print(concatenated)
```

Descriptive Statistics in Pandas

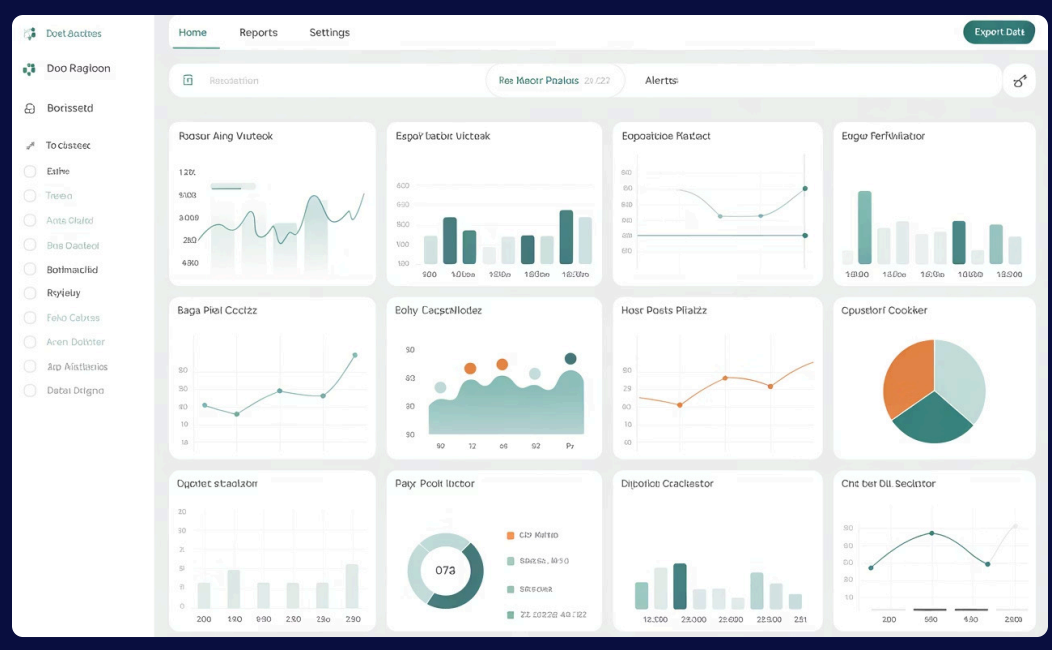
Basic Statistics

```
import pandas as pd
import numpy as np

# Sample DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50],
    'C': [100, 200, 300, 400, 500],
    'Category': ['X', 'Y', 'X', 'Y', 'X']
})

# Summary statistics for all numeric columns
print(df.describe())
#              A          B          C
# count  5.000000   5.000000   5.000000
# mean   3.000000  30.000000 300.000000
# std    1.581139  15.811388 158.113883
# min    1.000000  10.000000 100.000000
# 25%    2.000000  20.000000 200.000000
# 50%    3.000000  30.000000 300.000000
# 75%    4.000000  40.000000 400.000000
# max    5.000000  50.000000 500.000000

# Individual statistics
print(df['A'].mean())     # 3.0
print(df['B'].median())   # 30.0
print(df['C'].std())      # 158.11...
print(df['A'].min())      # 1
print(df['B'].max())      # 50
print(df[['A', 'B']].sum())
# A     15
# B    150
# dtype: int64
```



Grouping and Aggregation

```
# Group by a column and calculate statistics
grouped = df.groupby('Category')
print(grouped.mean())
#             A     B      C
# Category
# X         3.0  30.0  300.0
# Y         3.0  30.0  300.0

# Multiple aggregations
agg_result = grouped.agg({
    'A': ['min', 'max', 'mean'],
    'B': ['sum', 'mean'],
    'C': 'std'
})
print(agg_result)

# Apply different functions to all columns
multi_agg = grouped.agg(['count', 'mean', 'std'])
print(multi_agg)
```

Correlation and Covariance

```
# Correlation matrix
print(df.corr())
#      A    B    C
# A  1.0  1.0  1.0
# B  1.0  1.0  1.0
# C  1.0  1.0  1.0

# Covariance matrix
print(df.cov())

# Correlation between specific columns
print(df['A'].corr(df['B']))  # 1.0
```

Descriptive statistics are essential for understanding your data's central tendencies, spread, and relationships between variables. Pandas makes it easy to calculate these metrics across different subsets of your data.

Practical Data Example

Let's work through a practical example combining NumPy and Pandas to analyze sales data for an e-commerce business.

Creating the Dataset

```
import pandas as pd
import numpy as np

# Create product sales data
np.random.seed(42)  # For reproducibility

# Generate data
products = ['Laptop', 'Phone', 'Tablet', 'Monitor', 'Headphones']
categories = ['Electronics', 'Electronics', 'Electronics',
              'Accessories', 'Accessories']
stores = ['Online', 'Store A', 'Store B', 'Store C']

# Create 100 random sales records
n_records = 100
data = {
    'Product': np.random.choice(products, n_records),
    'Category': np.random.choice(categories, n_records),
    'Store': np.random.choice(stores, n_records),
    'Units': np.random.randint(1, 50, n_records),
    'Price': np.random.uniform(10, 1000, n_records).round(2)
}

# Create DataFrame
sales_df = pd.DataFrame(data)

# Add calculated column
sales_df['Revenue'] = sales_df['Units'] * sales_df['Price']

print(sales_df.head())
```



Analyzing the Data

```
# Get basic statistics
print(sales_df.describe())

# Top selling products by units
product_units = sales_df.groupby('Product')
['Units'].sum().sort_values(ascending=False)
print("Top selling products by units:")
print(product_units)

# Revenue by store and category
pivot = sales_df.pivot_table(
 values='Revenue',
 index='Category',
 columns='Store',
 aggfunc='sum'
)
print("\nRevenue by store and category:")
print(pivot)

# Filter for high-value transactions
high_value = sales_df[sales_df['Revenue'] > 5000]
print(f"\nHigh value transactions: {len(high_value)}")
print(high_value.head())

# Calculate percentage of sales by store
store_pct = sales_df.groupby('Store')['Revenue'].sum() /
sales_df['Revenue'].sum() * 100
print("\nPercentage of sales by store:")
print(store_pct.round(2))

# Create a new DataFrame with monthly aggregates
# (pretend we have a date column)
sales_df['Date'] = pd.date_range(
 start='2023-01-01',
 periods=len(sales_df),
 freq='D'
)
monthly = sales_df.set_index('Date').resample('M')[['Units',
'Revenue']].sum()
print("\nMonthly sales:")
print(monthly)
```

Combining NumPy and Pandas

NumPy and Pandas are designed to work seamlessly together. You can leverage NumPy's computational power with Pandas' data organization capabilities to create powerful data analysis workflows.

Converting Between NumPy and Pandas

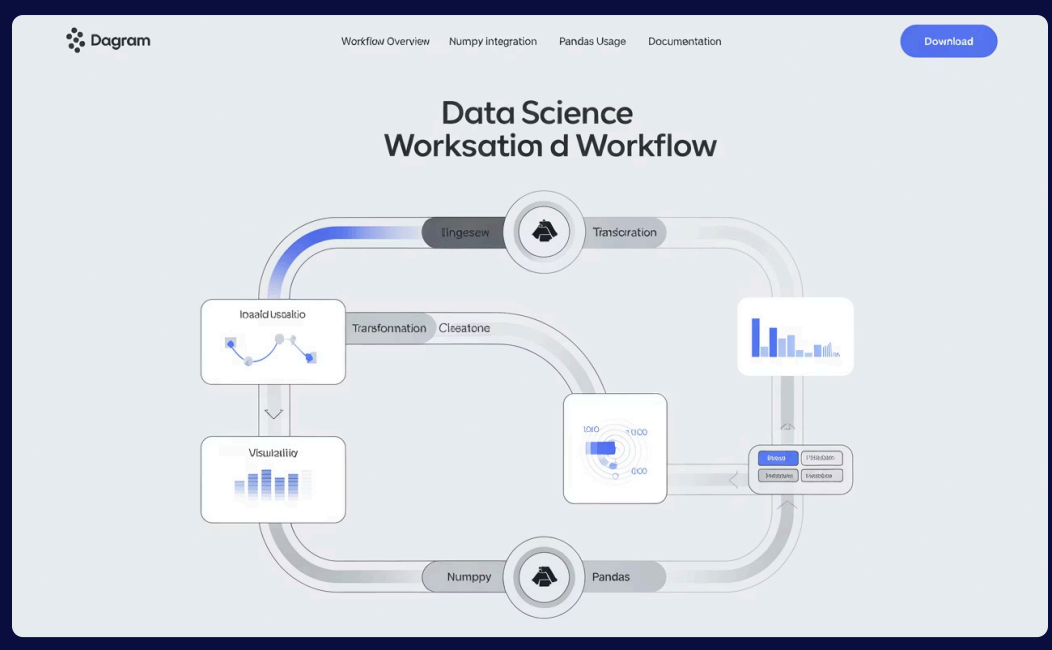
```
import pandas as pd
import numpy as np

# Create a DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50],
    'C': [100, 200, 300, 400, 500]
})

# Convert DataFrame to NumPy array
arr = df.to_numpy()
print(arr)
# [[ 1  10 100]
#  [ 2  20 200]
#  [ 3  30 300]
#  [ 4  40 400]
#  [ 5  50 500]]

# Convert single column to NumPy array
a_column = df['A'].to_numpy()
print(a_column)  # [1 2 3 4 5]

# Convert NumPy array back to DataFrame
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
new_df = pd.DataFrame(arr_2d, columns=['X', 'Y', 'Z'])
print(new_df)
#    X  Y  Z
# 0  1  2  3
# 1  4  5  6
# 2  7  8  9
```



Using NumPy Functions on Pandas Data

```
# Apply NumPy functions to DataFrame columns
print(np.sqrt(df['A']))
print(np.log(df['B']))
print(np.sin(df['C']))

# Apply NumPy functions to entire DataFrame
print(np.square(df))

# Use NumPy random to generate data for Pandas
random_data = np.random.normal(size=(100, 4))
random_df = pd.DataFrame(
    random_data,
    columns=['W', 'X', 'Y', 'Z']
)
print(random_df.head())
```

Advanced Combined Techniques

Custom Universal Functions

```
# Define a custom NumPy function
def custom_scale(x, factor=2):
    return np.where(x > 0, x * factor, x / factor)

# Apply to Pandas DataFrame
df['D'] = custom_scale(df['A'])
print(df)
```

Linear Algebra with Pandas Data

```
# Compute matrix product of two DataFrames
A = pd.DataFrame(np.random.rand(3, 3))
B = pd.DataFrame(np.random.rand(3, 2))
C = A.dot(B)
print(C)
```

Window Functions

```
# Rolling window calculations
df['rolling_mean'] = df['A'].rolling(window=3).mean()
print(df)
```

Conclusion & Next Steps

Key Takeaways

Foundation of Data Science

NumPy and Pandas form the bedrock of Python's data science ecosystem, providing efficient data structures and powerful analysis tools that underpin more advanced libraries.

Complementary Tools

NumPy excels at numerical operations with its array structure, while Pandas provides intuitive data handling with its DataFrame. Together, they create a complete data analysis toolkit.

Performance Optimization

Vectorized operations in NumPy and optimized methods in Pandas dramatically improve performance compared to vanilla Python, allowing you to work with larger datasets more efficiently.

What We've Covered

- Creating and manipulating NumPy arrays
- Advanced indexing, slicing, and broadcasting
- Vectorized operations and performance benefits
- Building and exploring Pandas DataFrames
- Data selection, filtering, and transformation
- Statistical analysis and visualization
- Practical data workflows combining both libraries



Recommended Next Steps

Explore Real Datasets

Apply what you've learned to public datasets from Kaggle, UCI Repository, or data.gov. Try analyzing COVID-19 statistics, housing prices, or stock market data.

Learn Data Visualization

Master Matplotlib and Seaborn to create compelling visualizations of your data analysis findings. Add interactive elements with Plotly.

Move to Machine Learning

Build on your NumPy and Pandas skills with scikit-learn to develop predictive models and advanced analytics solutions.

Final Thought

The time invested in mastering NumPy and Pandas pays enormous dividends across virtually all data-oriented programming tasks. These libraries have transformed Python from a general-purpose language into the world's most popular platform for data analysis and scientific computing.

Continue practicing with these tools regularly, and you'll develop the ability to transform raw data into meaningful insights with remarkable efficiency and clarity.